Spotify: Distributed Concepts Exhibited

Architecture, paradigms, algorithms and technologies, and where each one appears in the project

Line numbers refer to the final project zip (*MusicServer.java*, *Balancer.java*, *SpotifyWeb.java*, *SpotifyTerminal.java*, *app.js*). "Visible in" tells you how to see the concept working while running the app.

A. Distributed Architecture

Architecture feature	What it means here	Where it shows in the project
Client-server	Clients send requests; servers hold the data and reply.	Browser (*app.js fetch()*, line 14) and *SpotifyTerminal.java* are clients; the three *MusicServer.java* nodes are servers.
Gateway (multi-tier)	A browser cannot speak the raw socket protocol, so a gateway turns each HTTP */api/...* call into one socket command.	*SpotifyWeb.java*: *ROUTES* table (line 20) and *api()* (line 64). Path: Browser → SpotifyWeb → Balancer → node.
Symmetric peer cluster (no master)	Three identical nodes, each with a full copy of the data. Any node can serve any request, so there is no leader to lose.	*MusicServer.PORTS = {5001, 5002, 5003}* (line 22); *main()* starts all three nodes (line 401). Sidebar lists them.
Client-side load balancer	The balancer is a library inside each client, so there is no separate balancer process that could fail.	*Balancer.java*, used by both *SpotifyWeb.java* and *SpotifyTerminal.java*.
Replicated state + shared durable storage	Every node keeps the data in memory; writes are also saved to two text files that all nodes reload at start-up.	*users.db*, *userdata.db* in *server/data/*; *saveData()* (line 95); *loadUsers()* / *loadData()* called in the *MusicServer* constructor.
Request/response over short TCP connections	One socket per request: *COMMAND<TAB>args* in; *OK|port|clock*, data lines and *END* out.	*Balancer.ask()* (lines 34–48) on the client side; *MusicServer.serve()* (line 177) on the server side.

B. Distributed Paradigms

Paradigm	Where it is exhibited in the code	Visible in
1. Client-server	Clients talk to *MusicServer* nodes over *Socket* / *ServerSocket* with a request/reply protocol.	Every user action is one request and one reply.
2. Load balancing (round robin)	*Balancer.send()*, lines 16–19: *start = counter++ % n*.	Sidebar “last served” marker moves 5001 → 5002 → 5003; terminal prints “served by node …”.
3. Replication (write-all)	*MusicServer.handle()* (lines 190–206) applies a write, then *replicate()* (line 210) sends *REPL* to the other nodes. Writes are listed in *WRITES* (line 23).	Like a song; any node then shows the same Library.
4. Fault tolerance / failover	*Balancer.send()* catches the connection error (line 26) and tries the next node; *replicate()* skips a dead peer.	“Crash last node” button (*app.js* line 197) or *crash 5001* in the terminal; the app keeps working.
5. Logical time (Lamport clock)	*clock* field (line 29); updated in *handle()* (lines 196–200).	Terminal prints “logical clock N” (*SpotifyTerminal.java* line 129); backend log prints *clock=*.
6. Concurrency / multithreading	One thread per connection (*MusicServer.java* lines 163–167); thread pool in *SpotifyWeb.java* (line 32); *synchronized* protects shared data.	Several browser tabs plus the terminal can be used at once.
7. Stateless services	*NEXT* / *PREV* carry the current song id and playlist; the username travels with every request. No node stores a session.	Next / Previous work even when different nodes answer.
8. Persistence (durability)	*saveData()* after every write (line 202); loaded again at start-up.	Restart the backend: accounts, likes and playlists remain.

C. Distributed Algorithms and Mechanisms

Algorithm / mechanism	How it works	Where it shows in the project
Round-robin scheduling	A shared counter picks the next node: *start = counter++ % n*, so requests are spread evenly.	*Balancer.send()*, line 19; counter incremented inside *synchronized* (line 19).
Failover (retry on next node)	Loop over all nodes starting at *start*; on *IOException* log it and try the next. Error only if all nodes are down.	*Balancer.send()*, lines 20–30.
Write-all replication (read-one, write-all)	The receiving node applies the write, then forwards *REPL<TAB>clock<TAB>command* to every peer. A *REPL* message is not forwarded again, which prevents loops. Dead peers are skipped.	*handle()* (190–206), *replicate()* (lines 210–224), *WRITES* (line 23). Check *!fromPeer* on line 205.
Lamport logical clock	Local write: *clock++*. On *REPL* with clock c: *clock = max(clock, c) + 1*. The clock is returned in the reply header *OK|port|clock*.	*MusicServer.java* lines 196–200 and 206; shown by *SpotifyTerminal.java* line 129.
Failure detection (time-outs and ping)	A node that does not accept within 800 ms or reply within 4 s is treated as down; replication uses 500 ms / 1 s. *PING* is a health check.	*Balancer.java* lines 36–37 and *isUp()* line 50; *replicate()* lines 214–215; sidebar polling *app.js* line 198.
Failure simulation	The *CRASH* command closes the node's server socket, which behaves like a real node going down.	*MusicServer.serve()*, line 186; *crash* command in *SpotifyTerminal.java* line 57.
Mutual exclusion (critical sections)	A per-node lock around data changes; a static lock on the class for the files, because the three nodes share them.	*synchronized (this)* line 195; *synchronized (MusicServer.class)* lines 47, 63, 105, 117.
Atomic file update	Write to a temporary file, then move it over the real file, so a crash cannot leave a half-written file.	*saveData()*, lines 95–111 (*Files.move*, line 110).
Stateless request design	Each request holds everything the node needs, so any node can answer it.	*NEXT* / *PREV* commands; username sent with each request (*app.js*, *SpotifyTerminal.java*).

D. Distributed Technologies Used

Technology	Role in the project	Where it shows in the project
TCP sockets (*java.net.Socket*, *ServerSocket*)	All communication: client to node, node to node, and the web server itself.	*MusicServer.java* line 162; *Balancer.java* line 36; *SpotifyWeb.java* line 31.
Threads and thread pool	Thread per connection on each node; cached thread pool in the web server.	*MusicServer.java* lines 163–167; *SpotifyWeb.java* line 32 (*Executors.newCachedThreadPool()*).
Custom text protocol	Tab-separated commands, pipe-separated replies, *END* terminator, *REPL* for replication.	*Balancer.ask()*; *MusicServer.handle()*.
Hand-written HTTP server	Serves *public/*, JSON for */api/**, and audio with **HTTP Range** so seeking works. No framework.	*SpotifyWeb.java*: *staticFile()* line 132, *audio()* lines 145–166.
HTTP + *fetch()* + JSON	How the browser talks to the gateway.	*app.js* line 14 and the sign-up / login calls (lines 44, 55).
File-based persistence	Two UTF-8 text files, *users.db* and *userdata.db*.	*saveData()*, *loadData()*, *loadUsers()* in *MusicServer.java*.
SHA-256 hashing	Passwords are stored only as hashes.	*MusicServer.java* line 146 (*MessageDigest*).
Browser *localStorage*	Remembers the logged-in user across page refreshes.	*app.js* lines 9, 59, 71.
javax.sound.sampled	Audio playback in the terminal client (WAV only).	*SpotifyTerminal.java*, *Clip* (line 158).

Not used: no web framework, no message broker, no database server and no consensus library. Everything above is built directly on Java sockets, which is what makes each mechanism visible in a few lines of code.